

Optimization of R-tree Construction for Spatial Data Using GPU

by Nirmal Shahi, Bachelor of Science in Computer Science and Information Technology

A Thesis Submitted in Partial
Fulfillment of the Requirements
for the Degree of
Master of Science
in the field of Computer Science

Advisory Committee:

Mark McKenney, Chair

John Matta

Manas Jyoti Das

Graduate School
Southern Illinois University Edwardsville
December 2025

TABLE OF CONTENTS

LIST OF FIGURES	v
LIST OF TABLES	vi
Chapter	
1. INTRODUCTION	1
2. LITERATURE REVIEW	6
3. PRELIMINARY CONCEPTS	11
3.1 Introduction	11
3.2 Spatial Data and Multidimensional Indexing	11
3.3 R-tree Data Structure and Hierarchical Organization	12
3.4 Minimum Bounding Rectangle (MBR) and Overlap Concept	12
3.5 R-tree Variants and Their Characteristics	13
3.6 GPU Architecture for Parallel Computation	13
3.7 CUDA Programming Model and Memory Hierarchy	14
3.8 Parallelization Challenges in Tree-Based Structures	14
3.9 Performance Metrics for Spatial Index Evaluation	14
3.10 Methodology	15
4. BOTTOM UP R-TREE CONSTRUCTION ALGORITHM	17
4.1 Introduction	17
4.2 Concept Overview	17
4.3 Key Characteristics of Bottom-Up Approach	18
4.4 Workflow of Bottom-Up Construction	18
4.5 Algorithm	19
4.6 GPU Implementation Details	19
4.7 Illustrative Example	20
4.8 Advantages of the Bottom-Up GPU Design	20
5. EXPERIMENTAL EVALUATION OF BOTTOM UP R-TREE ALGORITHM	22
5.1 Introduction	22
5.2 Experimental Objectives	22
5.3 Experimental Setup	22
5.4 Evaluation Parameters	23
5.5 CPU vs GPU Performance Comparison	23

5.5.1	Detailed Interpretation of Results	24
5.5.2	Summary	24
6.	TOP DOWN R-TREE CONSTRUCTION APPROACH	26
6.1	Introduction	26
6.2	Concept and Principle	26
6.3	Algorithmic Overview	27
6.4	GPU Implementation	28
6.5	Illustrative Example	29
6.6	Advantages of the Top-Down GPU Design	29
6.7	Summary	30
7.	EXPERIMENTAL EVALUATION OF TOP DOWN R-TREE ALGORITHM	31
7.1	Introduction	31
7.2	Experimental Setup	31
7.3	Evaluation Metrics	31
7.4	CPU vs GPU Performance Comparison	32
7.4.1	Detailed Interpretation of Results	32
7.4.2	Summary	33
8.	SORT-TILE-RECURSIVE (STR) R-TREE CONSTRUCTION APPROACH	34
8.1	Introduction	34
8.2	Principle of the STR Construction	34
8.3	Algorithmic Workflow	35
8.4	GPU Implementation Details	36
8.5	Illustrative Example	38
8.6	Advantages of the STR GPU Design	39
8.7	Summary	39
9.	EXPERIMENTAL EVALUATION OF STR R-TREE ALGORITHM	40
9.1	Introduction	40
9.2	Experimental Objectives	40
9.3	Experimental Setup	40
9.4	Evaluation Metrics	41
9.5	CPU vs GPU Build Performance	41
9.5.1	Detailed Interpretation	41
9.5.2	Summary	43

10. QUALITY EVALUATION OF GPU R-TREE CONSTRUCTION	
ALGORITHMS	44
10.1 Introduction	44
10.2 Quality Metric Definitions	45
10.2.1 Node Utilization (U)	45
10.2.2 Coverage Ratio (C)	45
10.2.3 Sibling Overlap Ratio (O)	45
10.3 Experimental Methodology	46
10.4 Structural Quality Results	46
10.4.1 Summary Table With Real Measured Values	46
10.5 Detailed Analysis	47
10.5.1 Bottom-Up (Z-order) Quality	47
10.5.2 top-down Quality	47
10.5.3 GPU STR (Fully Parallel STR) Quality	48
10.6 Comparative Discussion	49
10.6.1 Which Algorithm Has the Best Spatial Quality?	49
10.6.2 Effect of Fanout	49
10.6.3 Practical Implications	49
10.7 Conclusion	50
11. RESULTS AND COMPARISONS	51
11.1 GPU Acceleration and Speedup Analysis	51
11.2 Quality Analysis	54
11.2.1 Performance Evaluation on Real TIGER Dataset	56
12. CONCLUSION AND FUTURE RECOMMENDATION	58
12.1 Conclusion	58
12.2 Future Recommendations	59
12.3 Closing Remarks	61
REFERENCES	62

LIST OF FIGURES

Figure		Page
11.1	CPU vs GPU Build Time for Bottom-Up, top-down, and GPU STR on Synthetic Datasets (1K–100K).	52
11.2	Speedup comparison across dataset sizes for all three algorithms.	53
11.3	Average sibling overlap ratio vs. fanout M for Bottom-Up, Top-Down, and GPU STR on 100,000 synthetic rectangles.	54
11.4	Average coverage ratio vs. fanout M for Bottom-Up, top-down, and GPU STR on 100,000 synthetic rectangles.	55
11.5	Average node utilization vs. fanout M for Bottom-Up, top-down, and GPU STR on 100,000 synthetic rectangles.	56

LIST OF TABLES

Table		Page
5.1	Bottom-Up R-tree Construction: CPU vs GPU Build Time and Speedup . .	23
7.1	Top-Down GPU R-tree Construction Results (RTX 6000 PRO Blackwell) . .	32
9.1	GPU STR R-tree Construction Results (RTX 6000 PRO Blackwell)	41
10.1	Measured Structural Quality Metrics for 100,000 Synthetic Rectangles	46
11.1	R-tree Construction Performance on Real TIGER Dataset	57

CHAPTER 1

INTRODUCTION

R-trees are tree-based data structures employed for spatial access methods. This is used for indexing multi-dimensional information such as geographical coordinates, rectangles or polygons. The use of efficient spatial indexing structure is elevated due to the increasing demand of spatial data in modern computing. Spatial data is important for various applications such as geographic information systems (GIS), multimedia databases, computer-aided design, urban planning and emerging technologies such as smart cities and automated vehicles. The main problem in managing spatial data is to support quick search, query operations, and retrieval by ensuring scalability for massive datasets. The R-tree is commonly used in managing spatial objects hierarchically using minimum bounding rectangles (MBRs) effectively among various multidimensional indexing structures. Since its proposal by Guttman (1984), the R-tree has been refined into a number of variants, such as the R + -tree (Sellis et al., 1987), which minimizes the redundancy of search paths, the R -tree (Beckmann et al., 1990), which optimizes the node splitting and reinsertion to get more efficient in a query, and the Hilbert R-tree (Kamel & Faloutsos, 1994), which uses space-filling curves to optimize packing of nodes. The main idea behind these structures is to ensure that there are fewer MBR overlaps and that the hierarchies are balanced which in turn decreases the number of nodes to be visited when making queries.

With the emergence of general-purpose computing on graphics processing units (GPGPU), the handling of spatial data has further changed. GPUs are very parallel structures built around thousands of simple cores that can execute huge amount of threads at the same time. In contrast to CPUs, which use a small number of powerful cores specialized to run sequential applications, GPUs can perform data-parallel application; therefore, they are better suited

to computationally intensive workloads such as graphics rendering, matrix multiplication, machine learning, and spatial indexing. Recent research indicates that the benefits of using GPU acceleration can improve the construction of an R-tree and query processing by up to an order-of-magnitude over CPU-based implementations (Nong et al., 2024). This speed is especially beneficial in GIS systems like real-time map overlays, route optimization and spatial joins that require millions of spatial objects to be indexed and queried in seconds. Furthermore, the space-filling curves which include Z-ordering curves and Hilbert curves improve parallelism because they convert multidimensional spatial coordinates to the one-dimensional value which makes spatially close objects to be close in the index (Kamel & Faloutsos, 1994). Subsequently, GPSUs in conjunction with novel indexing techniques can provide promising platform to real-time and large cumulative spatial data management.

Although these have been made, there is still a major difficulty in successfully transforming R-trees to the GPU architectures. The classic R-tree algorithms are sequential and CPU based, and are heavily based on insertion based building, recursive walks and pointer based memory maps. Such techniques are not efficient in case of GPUs, which have a Single Instruction Multiple Thread (SIMT) way of execution. A significant constraint is branch divergence: as threads in a GPU warp take different execution paths on a tree walk, some threads have to idle and this results in an extreme waste of the available computational units (Nong et al., 2024). Likewise non-coalesced or random accesses to memory e.g. pointer-based structures affect the performance negatively on the GPU since global memory can only be efficient when a structure is accessed in aligned and contiguous blocks. Such architectural incompatibilities render porting naively R-tree algorithms based on the CPUs to the GPUs very inefficient.

The other challenge is on the balancing of the speed and quality of trees during construction. Sequential insertion algorithms are slow in nature but they create better trees with less MBR overlaps. Conversely, the batch construction techniques are more quick however can

be inefficient with regard to query performance since nodes are packed inefficiently (Luo et al., 2012), (You et al., 2013). In case of large scale datasets with millions of objects, overly large overlaps cause unnecessary search paths, longer query times, and wasted gpu time. Scalability is also constrained by memory: GPUs generally have less memory than CPU systems, and node layouts that are fixed in size, as used in some earlier designs of the GPU R-tree (Luo et al., 2012), lead to waste when not all the nodes are used. Also, most of the current solutions are concerned with the individual query performance as opposed to processing batches of queries, which is a more realistic view in real-world applications like spatial joins or overlay operations. The coordination required by batch queries is across thousands of threads, and efficient use of memory in dynamically created query tasks and global coordination across tree levels, all of which are difficult to achieve at the same time in GPUs.

Also, space-filling curves are proven to be better at node packing, but they do not fit into the construction done by a GPU without difficulties. Hilbert curves clustering is better than Z-ordering, however, its computation is more complicated and less amenable to computing on a GPU. This is a tradeoff of computational efficiency against preservation of spatial locality, an additional dimension to the problem. Other than construction and query, update and deletion operations are nevertheless deemed inappropriate to execute on a GPU because of the overheads of synchronization and the concurrency issues, which require an entire rebuilding of the tree at every single tree modification. This is a deficit of dynamic adaptability, which restricts the usefulness of the R-trees that are accelerated using GPUs in the real-time context where data is actively updated. Collectively, these issues highlight the importance of new GPU-aware designs, which can both deal with parallelism, memory-efficiency, construction quality, and query throughput.

This thesis addresses these challenges by developing a suite of optimized GPU-based strategies for R-tree construction that effectively scale to large spatial datasets. The work fo-

cuses on designing GPU-centric data structures and kernels that maximize memory coalescing, minimize warp divergence, and fully exploit fine-grained parallelism across tree levels. Three complementary algorithms are investigated: a Bottom-Up method that employs Z-order (Morton) space-filling curves to generate locality-preserving clusters; an improved top-down approach that recursively partitions spatial objects in two dimensions to minimize coverage and overlap; and a fully parallel GPU STR algorithm that performs strip-based sorting and hierarchical MBR reduction entirely on the GPU. Together, these methods provide a spectrum of performance–quality trade-offs. By integrating insights from all three approaches, the thesis aims to achieve both high construction throughput and high-quality spatial hierarchies, ensuring that the resulting R-trees deliver fast build times while significantly improving query performance on modern GPU hardware.

Furthermore, the thesis investigates the use of real-world datasets, including TIGER/Line shapefiles from the U.S. Census Bureau, which provide large-scale road, county, and boundary data essential for spatial experiments. These datasets, combined with synthetic benchmarks, enable a comprehensive evaluation of the proposed methods across different scales and spatial distributions. Performance comparisons are conducted between sequential CPU-based R-trees, and the newly developed algorithms, with emphasis on build time and overlap metrics. The results demonstrate that the proposed GPU-based R-tree achieves significant speedups construction, often approaching an order of magnitude improvement over CPU baselines. Importantly, the algorithms also preserve or enhance tree quality, leading to fewer node accesses during queries.

In addition to the performance improvement, this study adds to the comprehension of the way the use of the GPU architecture can be systematically applied to spatial indexing. Through tradeoff analysis between bottom-up, top-down and STR construction, the thesis provides a guideline in selecting the algorithm to use depending on the size of the dataset, the spatial distribution and the query workload. Limitations that are also emphasized in the

work include the inefficiencies in updates and suggest some possible paths of extending the framework to the multi-gpu or hybrid cpu-gpu settings. Finally, the research of this thesis benefits the state of the art in the field of GPU-accelerated spatial indexing by providing the practical solutions in the sphere of GIS, urban data analysis, and other fields where it is essential to organize the management of the large-scale spatial data effectively.

CHAPTER 2

LITERATURE REVIEW

The initial philosophy of the R-tree was proposed by Guttman (1984) as a dynamic index structure that can be used to manage multidimensional space information. The R-tree is a spatial data structure that organizes spatial objects based on Minimum Bounding Rectangles (MBRs) and can add, delete and query items without rebuilding the structure. This is because its balanced tree structure achieves the fastest logarithmic search time in the best-case scenario, and it is an easy option to use with spatial queries like range queries and nearest-neighbor queries. Nevertheless, the original design by Guttman has the disadvantage of overlapping MBRs between sibling nodes and this tends to give rise to multiple search paths and deteriorate query performance with an increase in datasets. Irrespective of this drawback, the R-tree has become the foundation of many other techniques in indexing, with many variants proposed since that time, seeking to eliminate redundancy, better node splitting algorithms, and generally more efficient queries. This is why the work of Guttman can still be very influential and can serve as the basis under which adaptations based on GPU acceleration, including those discussed in this thesis, have been built.

To eliminate the issue of overlapping bounding rectangles, Sellis et al. (1987) suggested R+ tree as an improvement of the original R-tree. The R + tree removes overlapping areas between the siblings of a given spatial object by placing the spatial object in only one leaf node by using object separation techniques. This enhancement eliminates query ambiguities greatly and enhances quicker search operations specially on point queries. The mechanism of separation, however, involves the replication of objects in a variety of nodes and results in inefficiency in storage and increased costs of updating it. Moreover, although the R+-tree is

better at accuracy of query, it cannot perform well in dynamic updates and huge datasets where duplication increases memory needs. Nevertheless, the R+-tree does not negate the place of the R+ tree in the development of spatial indexing in pointing out the tradeoff between query accuracy and storage efficiency. Principles of reducing overlap created by this work are still used in modern methods, such as GPU-based implementations, where memory-efficiency and query-throughput are of paramount importance.

The R*-tree was created by Beckmann, Kriegel, Beckmann et al. (1990), and it added massive optimization to the original R-tree, but was mostly geared towards minimizing overlap and space wastage. The R*-tree uses sophisticated node-splitting algorithms, and a forced reinsertion mechanism which does a better redistribution of entries during insertions. Such enhancements reduce the area, overlap and margin values of MBRs and consequently, optimize query performance of range searches and nearest-neighbor queries. Experimental tests proved that R-tree is always better than R+ and R-tree in both storage use and speed of query. However, these enhancements are at the cost of adding to the complicatedness of insertion, which can adversely affect the performance of construction with large datasets. Nevertheless, in spite of this, the R*-tree has become the default spatial indexing choice because it is the best trade-off between accuracy and efficiency. Its focus on query performance makes it especially important to GPU implementations where query batching and overlap reduction are important factors to consider.

One of the earliest efforts to develop R-trees to GPUs was made by Luo et al. (2012), which proposed a parallel implementation that relied on breadth-first search (BFS). Their design placed R-tree nodes in a linear array, which made access to coalesced memory and efficient traversal by the GPU. Their implementation allowed parallel searching speed by combining queues with array-based designs as opposed to the traditional CPU-based designs. There were

however two serious limitations to the method: each R-tree node was fixed in size irrespective of its occupancy, which caused a substantial amount of memory to be go to waste, and the method was not level-wise accessable, which restricted the ability to scale to large sizes. Moreover, traversal algorithms were simply made based on CPU-based algorithms, which were not utilizing the full parallelism of GPUs. Although innovative, this work emphasized the challenge of redesigning R-trees to be used by GPUs, instead of simply moving algorithms that run on CPUs. The information presented by this paper inspired subsequent GPU-aware design, such as those discussed by this thesis.

Kim et al. (2022) introduced the LBPG-tree, a multi-GPU indexing framework designed to maximize parallelism for high-dimensional range queries by compact-and-sort scheduling of candidate nodes and hierarchical pipelining across GPUs. The design explicitly exploits GPU L2 cache locality, reducing random accesses during concurrent searches, and orchestrates inter-GPU workloads to keep streaming multiprocessors saturated. Experiments showed LBPG-tree outperformed prior GPU-aware indexes (e.g., G-tree) and scaled across multiple devices, highlighting how batching, node compaction, and GPU-conscious memory layouts jointly lift throughput. Although the focus is high-dimensional data rather than 2D/3D GIS, two lessons generalize: (i) compact/sort schemes materially improve coalescing and cache reuse; (ii) hierarchical pipelines mitigate synchronization stalls across deep traversals. For R-trees on GPUs, these ideas translate to level-wise task compaction and cooperative scheduling for breadth-first batched queries. This thesis borrows such scheduling principles to improve R-tree batch traversal and explores when multi-GPU task partitioning further increases throughput for spatial workloads.

Xiao et al. (2022) proposed an RDMA-enabled in-memory computing platform (“Catfish”) for distributed R-tree processing, addressing imbalances between server overload and under-

utilized client/network resources. They designed two RDMA mechanisms: a fast messaging path that bypasses CPU intervention for request handling, and an in-place data path that minimizes copies for index/node access. The system adaptively balances workloads across client and server resources and achieves low latency and high throughput on clusters. While architecturally different from single-GPU solutions, this work underscores the performance gains from network-aware memory access and from decoupling control from data movement. For GPU R-trees, the analogy is to decouple kernel scheduling from bulk data transfers and to pre-stage level ranges, enabling overlap of PCIe/NVLink movement with batched breadth-first processing. The study also motivates hybrid CPU–GPU deployments where the GPU handles parallel query/filter phases while RDMA pipelines distribute work at cluster scale—an avenue for multi-GPU future work in this thesis.

Geng et al. (2024) exploited modern GPUs’ ray-tracing hardware (e.g., NVIDIA RT Cores) to accelerate polygon-level spatial joins. RayJoin reframes spatial predicates as ray-geometry intersection queries over hardware-accelerated bounding-volume hierarchies, delivering substantial speedups versus plane sweep and grid/tree-assisted algorithms. The key insight is leveraging fixed-function traversal/intersection engines to prune candidate pairs and evaluate containment/overlap with high throughput. Although RayJoin targets join semantics rather than general R-tree indexing, its results reaffirm the value of hardware-friendly hierarchies (e.g., well-packed BVHs) and batch-oriented traversal. For GPU R-trees, two implications follow: (i) breadth-first, warp-cooperative traversal reduces divergence; (ii) building indices that map cleanly onto hardware traversal patterns (tight MBRs, low overlap) yields disproportionate gains. This thesis applies these principles when designing bottom-up/top-down R-tree builders that minimize overlap and favor contiguous, level-addressable layouts to support RT-like coherent traversal on standard CUDA cores.

Zhang et al. (2023) introduced RUMMY which is a GPU-based query engine that can be used to serve vectors queries at a high-performance level even with a dataset that is larger than the memory of the GPU. The system has implemented reordered pipelining to synchronize host-to-device transfers with the compute, and dynamically reorders work during execution so that transmission and kernel execution are balanced. Though RUMMY is a search method based on the similarity of vectors as opposed to searching based on space, the fundamental problem of the system, which is the ability to use the GPU effectively with limited device-memory resources, resembles large GIS workloads. The contributions of the paper (chunked processing, adaptive reordering, and overlapped I/O-compute) can be successfully applied to the construction of R-trees on a practically large scale (polygons/MBR sets) and batched queries. Considering this thesis, we follow these same pipelining and chunk sizing to maintain throughput when searching or accessing TIGER-scale data, so that the expansion, compacting and filtering of the R-tree levels can continue without stalling on PCIe/NVLink bottlenecks or chip-memory constraints.

CHAPTER 3

PRELIMINARY CONCEPTS

3.1 Introduction

The chapter contains the key ideas of the proposed R-tree framework based on GPUs. It provides the theoretical basis needed to acquire the understanding of the experimental methodologies that will be discussed in subsequent chapters. Such issues as the nature of spatial data, multidimensional indexing, the structure of R-tree, the architecture of the GPU and the performance measure are explained. Making these concepts work together allows the mapping of spatial algorithms onto parallel hardware to be done efficiently.

3.2 Spatial Data and Multidimensional Indexing

Spatial data are geometric constructs like points, lines and polygons which have one or more dimensions. These physical features such as roads, rivers and administrative boundaries in geographic information systems (GIS) are equated to these entities. These data need special indexing structures capable of managing such data and support of spatial queries such as range search, nearest neighbor, and spatial joins. The conventional indexes used (e.g. B-trees, hash indexes) cannot handle multidimensional data since they rely on linearity. Therefore, to maintain spatial associations, multidimensional indexing designs, including R-trees, KD-trees, Quad-trees, and Grid indexes have been created. These structures store spatial objects hierarchically, whereby query operations can be done with logarithmic time. The most popular of them is the R-tree as it has dynamic properties, supports arbitrary geometrical shapes, and supports the work with point and area objects.

3.3 R-tree Data Structure and Hierarchical Organization

R-tree was developed by Guttman (1984) and is a height-balanced tree based on the B-tree but to support multidimensional objects. An R-tree node has several entries and each entry has a pointer to an entry in the child node and an associated Minimum Bounding Rectangle (MBR) which describes all the spatial objects contained in the child node. The direct references to the actual spatial objects are stored in the leaf nodes and the aggregated MBRs are stored in the internal nodes. The root node is a representation of the whole spatial domain. Only the nodes with which the query region intersects are accessed during queries thereby greatly limiting the search space. The height of the tree increases logarithmically with its numbers, and the node capacities are based on a specified parameter M , which is the maximized number of entries to each node. The hierarchical design is also balanced in search directions, yet efficiency is greatly dependent on how much the MBR overlaps between sibling nodes are reduced.

3.4 Minimum Bounding Rectangle (MBR) and Overlap Concept

The MBR is an essential geometric abstraction which is an expression of the smallest possible rectangle that contains a spatial object or group of objects. It is represented by two coordinates, that is, the bottom-left (x_0, y_0) and the top-right (x_1, y_1) points. The calculation of each node MBR is done to their child MBRs in R-tree construction. There is however a problem with overlapping MBRs that makes query operations redundant as two or more paths can match a query condition. Reduction of overlaps is thus important in ensuring the maximization of query performance. Some of the heuristics include area enlargement, perimeter, and minimize overlaps that are used in the process of insertion and splitting to ensure that MBRs are compact. In R-trees on a GPU, the difficulty lies in encoding these heuristics into parallelizable algorithms, which do not cause divergence between threads and retain spatial proximity.

3.5 R-tree Variants and Their Characteristics

Several variants of R-trees have been developed over time to enhance efficiency and memory use when conducting a query. The R+ tree (Sellis et al., 1987) removes overlapping areas between sibling nodes by imposing on them the object partitioning but at the expense of redundancy. The R-tree is improved by the R-tree+ (Beckmann et al., 1990), which re-Emily splits node sigma, causing the output of small MBRs and faster query performance. To linearize space in multidimensional space and enhance locality of data, the Hilbert R-tree (Kamel & Faloutsos, 1994) employs space-filling curves to linearize space. Recent GPU-based versions, e.g. the Parallel R-tree (Nong et al., 2024) and the G-tree (Kim et al., 2022), are concerned with the reorganization of memory layout to support coalesced access and parallel traversal. Every variant is a tradeoff between space overhead, complexity and performance. These principles are applied in this thesis to develop bottom up and top down GPU algorithms that trade speed and quality of the tree.

3.6 GPU Architecture for Parallel Computation

Graphics Processing Unit (GPUs) are highly parallel processors that are no longer used to perform graphics but find significant application in general-purpose computation (GPGPU). A GPU is a cluster of hundreds of Streaming Multiprocessors (SMs) each with hundreds of CUDA cores that are able to execute thousands of threads at the same time. In contrast to CPUs, which are better balanced towards sequential process execution, GPUs are better suited to data-parallel workloads where the same calculation is done on large volumes of data. Parallelism model adheres to Single Instruction Multiple Thread (SIMT) execution model with threads being grouped in warps of 32 threads on average. Warp performs the same instruction at a single time, and thus permits high throughput, although specific attention must be paid to algorithm design in order to avoid divergent execution paths.

3.7 CUDA Programming Model and Memory Hierarchy

CUDA programming model also offers an abstraction to execute parallel algorithms using NVIDIA GPUs. Kernels are functions run on the GPU and are launched in a grid of blocks which have numerous threads. Shared memory can be used to share data between threads in the same block and global memory can be accessed by all threads but has a higher latency. CUDA programming effectively needs coalesced memory access, i.e. consecutive threads access neighboring memory addresses, which fully exploits bandwidth. There are also other memory spaces such as constant memory (read-only, cached), and texture memory (spatially localized). In the design of R-tree algorithms, it is desirable to lay nodes in adjacent memory arrays to encourage coalesced accesses and reduce global memory random accesses.

3.8 Parallelization Challenges in Tree-Based Structures

R-trees are tree-based data structures that are hierarchical in nature and have dependencies between parent and child nodes making parallelization difficult. On GPUs, issues are branch divergence, where threads in a warp execute different control paths; load imbalance, which is due to uneven data placement; and memory scattering, which is due to non-coalesced accesses to random node addresses. In order to counterfeit these problems, the algorithms should be configured to use level-wise or batch-processing, and in this approach, nodes at the same level of processing are executed simultaneously. Methods of bottom up and top down construction suggested in this thesis are aimed at taking advantage of these level-wise parallelism. Additionally, breadth-first traversal is preferred over depth-first traversal for query operations, as it aligns better with GPU architecture.

3.9 Performance Metrics for Spatial Index Evaluation

Evaluating spatial indexes requires multiple performance metrics to balance speed and structural quality.

- Construction Time: Measures the total time taken to build the R-tree, an indicator of algorithmic efficiency.
- Query Time: Evaluates search performance for range and overlap queries.
- Overlap Ratio: Represents the extent of MBR intersection between sibling nodes, directly affecting query efficiency.
- Coverage Ratio: Indicates how well MBRs represent underlying spatial data.

In subsequent chapters, these metrics are used to quantify the benefits of GPU acceleration and space-filling curve ordering on R-tree performance.

3.10 Methodology

The methodology of this study integrates both theoretical design and experimental implementation to evaluate the performance of R-tree construction on CPU and GPU platforms. The research follows a comparative experimental approach involving four major stages: dataset acquisition, preprocessing, R-tree construction, and performance evaluation. Synthetic rectangular datasets and real-world GIS data from the TIGER shapefiles were used to test scalability and spatial diversity. Each spatial object was converted into its Minimum Bounding Rectangle and encoded using either Z-order or Hilbert space-filling curves to establish spatial locality before construction. The CPU baseline employed the Guttman sequential insertion algorithm with OpenMP for limited parallelism, while the GPU version was implemented in CUDA C++ using a bottom-up parallel strategy optimized for coalesced memory access through array-based node representation. Experiments were conducted on an NVIDIA TITAN V and Intel Xeon CPU under identical parameters, measuring build time, query time, overlap ratio, and memory utilization. Both implementations were validated for consistency and accuracy across multiple dataset sizes. This methodological framework

ensures reproducibility, fair comparison, and a solid foundation for analyzing the efficiency gains of GPU-based R-tree construction using different space-filling curves.

CHAPTER 4

BOTTOM UP R-TREE CONSTRUCTION ALGORITHM

4.1 Introduction

This chapter presents the detailed implementation and performance analysis of the bottom-up R-tree construction method using both CPU and GPU platforms. The experiments evaluate how space-filling curve ordering (Z-order) and hardware parallelization (CPU vs GPU) influence the overall construction efficiency, memory utilization, and tree quality. The bottom-up approach was selected for its inherent parallelism, which aligns well with the GPU’s many-core architecture. By constructing R-trees level-by-level, large datasets can be processed concurrently, thereby accelerating build times. The experiments were conducted on both synthetic and real-world GIS datasets, and performance metrics such as build time, speedup, overlap ratio, and query efficiency were recorded.

4.2 Concept Overview

The bottom-up approach constructs the R-tree hierarchically from the leaf level upward, rather than inserting each spatial object one by one as in the traditional Guttman insertion algorithm. This design eliminates recursive insertions and sequential dependencies, making it ideally suited for parallel GPU computation.

In this method, all spatial objects (or their Minimum Bounding Rectangles, MBRs) are first sorted according to a space-filling curve (SFC) such as the Z-order or Hilbert curve, which preserves spatial proximity. Then, consecutive objects are grouped into leaf nodes, and higher levels of the tree are constructed by repeatedly merging adjacent groups until only the root node remains.

This level-wise construction ensures that all nodes at a given level are independent and can be built in parallel, enabling massive speedups on GPU architectures.

4.3 Key Characteristics of Bottom-Up Approach

- **Parallelization-Friendly:** Nodes at the same level are constructed independently, allowing GPU threads to process them concurrently.
- **Deterministic Structure:** Given a sorted input and fixed node capacity, the resulting tree structure is deterministic, unlike insertion-based trees.
- **Efficient Memory Layout:** Node data are stored in linear arrays (entry[], start[], end[], and level[]) to ensure coalesced global memory access.
- **Reduced Branch Divergence:** The method avoids conditional branching common in insertion algorithms, maximizing GPU throughput.
- **Flexibility:** Compatible with different ordering methods (Hilbert or Z-curve), allowing control over spatial clustering and query performance.

4.4 Workflow of Bottom-Up Construction

The construction process involves four major phases:

- **Phase 1: MBR Generation**

Each spatial object (polygon or rectangle) is converted into its Minimum Bounding Rectangle defined by (x_min, y_min, x_max, y_max).

- **Phase 2: Spatial Ordering (SFC Mapping)**

Each MBR is encoded using a Z-order or Hilbert key to represent its position in one-dimensional space while preserving spatial locality.

The dataset is then sorted according to these keys.

- **Phase 3: Leaf Node Formation**

Sorted MBRs are divided into groups of size M (node capacity). Each group forms a leaf node, and its parent MBR is computed as the union of all child MBRs.

- **Phase 4: Hierarchical Merging**

The process repeats recursively: adjacent MBRs from the previous level are grouped and merged into higher-level nodes.

This continues until a single root node remains.

4.5 Algorithm

Algorithm 1 GPU-Based Bottom-Up R-Tree Construction Using Z-Order Curve

Input: Dataset $D = \{O_1, O_2, \dots, O_n\}$, node capacity M

Output: R-tree stored in arrays: entry[], start[], end[], level[]

foreach object $O_i \in D$ *in parallel* **do**

 | Compute $MBR_i = (x_{min}, y_{min}, x_{max}, y_{max})$

end

foreach MBR_i *in parallel* **do**

 | Compute Morton key $K_i = ZOrderEncode(MBR_i.center)$

end

Sort all MBR_i by K_i using GPU parallel sort **while** $nodeCount > 1$ **do**

 | Group MBRs in blocks of M Compute parent MBRs as unions of child nodes Update
 | entry[], start[], end[], level[] arrays

end

return Root node entry[0] as final R-tree

4.6 GPU Implementation Details

The above algorithm is implemented using CUDA kernels where:

- **Kernel 1:** Computes MBRs and SFC keys for each object in parallel.
- **Kernel 2:** Performs a parallel radix sort on SFC keys (using the Thrust library).
- **Kernel 3:** Groups sorted MBRs into nodes and merges them into parent nodes.

The use of linear arrays for storing entries ensures contiguous memory layout, enabling coalesced global memory access that is critical for achieving high throughput on GPUs.

Synchronization between levels is handled at the kernel-launch boundary, ensuring that all child nodes are built before parent levels begin.

4.7 Illustrative Example

Consider a dataset of 16 rectangles and a node capacity of $M = 4$.

1. Each rectangle is assigned a Z-order key and sorted accordingly.
2. The first four rectangles form **Leaf Node 1**, the next four form **Leaf Node 2**, and so on.
3. The Minimum Bounding Rectangles (MBRs) of these leaf nodes are then merged into parent nodes. for instance, *Node 1-4* are merged to form **Parent Node A**.
4. The process continues iteratively until only one **root node** remains, representing the entire dataset.

The resulting tree will have a height given by:

$$h = \lceil \log_M(N) \rceil$$

where N denotes the total number of spatial objects. Thus, for $M = 4$ and $N = 16$, the resulting R-tree has a height of $h = 2$.

4.8 Advantages of the Bottom-Up GPU Design

The bottom-up GPU-based R-tree construction approach offers several key advantages as summarized below:

- **Parallel MBR Computation:** Each GPU thread independently computes the Minimum Bounding Rectangle (MBR) of one spatial object, enabling massive concurrency and significantly accelerating preprocessing.

- **Parallel Space-Filling Curve Sorting:** The use of the CUDA Thrust library allows $O(N \log N)$ parallel sorting of objects based on their Z-order keys, improving data locality and reducing sorting time.
- **Level-Wise Merging:** Nodes at the same level are merged concurrently, reducing synchronization overhead and allowing efficient level-by-level construction of the R-tree.
- **Array-Based Node Layout:** The R-tree is stored in contiguous arrays (`entry[]`, `start[]`, `end[]`, and `level[]`), which promotes coalesced global memory access and minimizes latency from random reads.
- **No Recursive Calls:** The algorithm avoids recursion, thereby eliminating GPU stack limitations and allowing more threads to execute in parallel without overflow risks.
- **Deterministic Build:** The construction process is deterministic for a given dataset and node capacity, ensuring reproducible results that make the method reliable for benchmarking and comparative evaluation.

CHAPTER 5

EXPERIMENTAL EVALUATION OF BOTTOM UP R-TREE ALGORITHM

5.1 Introduction

This chapter presents the experimental evaluation of the proposed bottom-up GPU-based R-tree construction algorithm using the Z-order curve. The experiments are designed to analyze the performance differences between CPU and GPU implementations and to investigate how varying the grid size (or thread block size) affects the overall build time and efficiency. The analysis focuses on identifying the optimal GPU configuration for constructing R-trees efficiently on large-scale spatial datasets.

5.2 Experimental Objectives

The objectives of this experimental evaluation are summarized as follows:

- To compare the build performance of CPU-based and GPU-based bottom-up R-tree construction.
- To quantify the relationship between data size, build time, and GPU resource utilization.
- To validate the efficiency and stability of the GPU implementation across different dataset scales.

5.3 Experimental Setup

All experiments in this research were executed on an **NVIDIA RTX PRO 6000 Blackwell Workstation Edition** GPU. This Blackwell-architecture device provides exceptionally high parallel throughput, large memory bandwidth, and ample VRAM capacity, making it well suited for large-scale spatial indexing and massively parallel R-tree construction. All GPU kernels were compiled using **CUDA 12.3** with full `-O3` optimization. The implementations

were written in **C++17** and relied on **CUDA Thrust** for parallel sorting as well as custom CUDA kernels for Morton key generation, MBR computation, strip-based spatial partitioning, and hierarchical node merging. This environment ensured a consistent, high-performance execution platform for evaluating all three R-tree construction algorithms developed in this thesis.

The experiments were performed using two datasets:

1. **Synthetic Dataset:** Randomly generated rectangles ranging from 10,000 to 2,000,000 objects.
2. **TIGER/Line Dataset:** Real-world U.S. Census Tiger Polygon dataset 2025 representing counties. <https://www2.census.gov/geo/tiger/TIGER2025/>

5.4 Evaluation Parameters

The following parameters were measured and analyzed:

- **Build Time (s):** Total time required to construct the R-tree.
- **Speedup:** Ratio of CPU build time to GPU build time.

5.5 CPU vs GPU Performance Comparison

Table 5.1 summarizes the build time comparison for CPU and GPU implementations.

Table 5.1: Bottom-Up R-tree Construction: CPU vs GPU Build Time and Speedup

Dataset	Size (N)	CPU Time (ms)	GPU Time (ms)	Speedup
Synthetic	1,000	0.170679	0.534026	0.319×
Synthetic	10,000	2.68633	0.669271	4.013×
Synthetic	100,000	34.514	1.99344	17.3138×
TIGER Block	185,976	41.258	1.46047	28.2499×

5.5.1 Detailed Interpretation of Results

Case 1: Synthetic Dataset (1,000 Rectangles). At this small scale, GPU execution is slower than CPU, with a speedup of only $0.319\times$. The dataset is too small to saturate GPU parallelism, and kernel launch overhead dominates the computation, making the sequential CPU implementation faster.

Case 2: Synthetic Dataset (10,000 Rectangles). The GPU begins to outperform the CPU significantly, achieving a $4.013\times$ speedup. At this size, parallel Morton key computation, sorting, and MBR reduction fully utilize the GPU’s parallel architecture, while the CPU shows near-linear scaling.

Case 3: Synthetic Dataset (100,000 Rectangles). This dataset demonstrates a dramatic GPU advantage, with a speedup of $17.31\times$. The GPU efficiently handles large-scale Z-order computation and parallel sorting, while the CPU’s build time increases substantially. This marks a strong scalability advantage for the GPU.

Case 4: TIGER Block Dataset (185,976 Objects). The real GIS dataset yields the highest acceleration, with a $28.25\times$ speedup. The GPU benefits from higher occupancy and efficient reduction operations on irregular, real-world spatial data. The CPU, in contrast, suffers from increased computation due to complex polygon shapes and larger bounding boxes.

5.5.2 Summary

Overall, the Bottom-Up GPU implementation consistently outperforms the CPU for all medium and large dataset sizes. While GPU overhead limits performance for very small inputs, the GPU becomes increasingly effective as dataset size grows. The TIGER dataset demonstrates the strongest speedup, confirming that the GPU-based Bottom-Up approach is highly suitable for large-scale and real-world spatial indexing workloads. This makes

Bottom-Up the fastest of all evaluated construction methods, offering excellent build-time performance with minimal overhead.

CHAPTER 6

TOP DOWN R-TREE CONSTRUCTION APPROACH

6.1 Introduction

This chapter presents the top-down R-tree construction approach implemented in both CPU and GPU environments. Unlike the bottom-up approach, which aggregates objects hierarchically from the leaf level upward, the top-down method recursively partitions spatial objects from the root downward. This approach offers flexibility in controlling node distribution and enables efficient handling of irregular spatial data distributions. The top-down method is well-suited for parallelization because each node subdivision can be executed independently, making it particularly advantageous for GPU architectures.

6.2 Concept and Principle

The top-down GPU R-tree construction begins by computing the global Minimum Bounding Rectangle (MBR) that covers all spatial objects in the dataset. The longer side of this global MBR is then selected as the split axis (x or y), and each object is assigned a one-dimensional key equal to the coordinate of its rectangle centre along this axis. A single global stable sort is performed on the GPU using these centre keys, producing a 1D ordering of all objects that roughly preserves spatial proximity along the dominant axis. After this initial sort, the algorithm proceeds in a purely top-down fashion by recursively partitioning contiguous ranges of the sorted array. At each level, every current node is represented as an index interval $[s, e)$ over the sorted objects. If the number of objects in the interval is less than or equal to the leaf capacity L , the interval is marked as a leaf node. Otherwise, the interval is split into up to M child ranges of (approximately) equal size, where M is the node fanout. This process is repeated level by level until no range contains more than L objects. Node MBRs for each range are computed on the GPU by a reduction kernel that unions the

rectangles within the associated index interval.

In contrast to the bottom-up design, which repeatedly merges child MBRs from the leaves upward, this top-down approach fixes a single global ordering of objects and refines the tree by successively subdividing index ranges. This strategy avoids repeated re-sorting at deeper levels and produces a balanced hierarchy whose nodes correspond to contiguous segments of the globally sorted array, making it naturally well suited to GPU execution.

6.3 Algorithmic Overview

The following pseudocode outlines the top-down construction process using spatial partitioning.

Algorithm 2 GPU-Based Top-Down R-Tree Construction with Single Global Axis Sort

Input: Dataset $D = \{O_1, \dots, O_N\}$ with MBRs, node fanout M , leaf capacity L

Output: R-tree leaves represented as index ranges over a globally sorted array

Compute global MBR of all objects in D Select split axis $a \in \{x, y\}$ as the longer side of the global MBR **foreach** object $O_i \in D$ *in parallel on GPU* **do**

 | Compute centre coordinate c_i of O_i along axis a

end

Copy rectangles to device array **directs** and keys c_i to **d_keys** Perform a single global stable sort of **directs** by **d_keys** Initialize current level ranges $\mathcal{R} \leftarrow \{[0, N)\}$ **while** $\mathcal{R}$ is not empty **do**

$\mathcal{R}_{\text{next}} \leftarrow \emptyset$ **foreach** range $[s, e) \in \mathcal{R}$ **do**

$\text{cnt} \leftarrow e - s$ **if** $\text{cnt} \leq L$ **then**

 | Mark $[s, e)$ as a leaf node Append $[s, e)$ to $\mathcal{R}_{\text{next}}$

else

$\text{groups} \leftarrow \lceil \frac{\text{cnt}}{M} \rceil$ $\text{groups} \leftarrow \min(\text{groups}, M)$ $\text{base} \leftarrow \lfloor \frac{\text{cnt}}{\text{groups}} \rfloor$ $\text{rem} \leftarrow \text{cnt} \bmod$

groups $\text{cur} \leftarrow s$ **for** $g \leftarrow 1$ **to** groups **do**

 | $\text{take} \leftarrow \text{base} + (g \leq \text{rem} ? 1 : 0)$ Append range $[\text{cur}, \text{cur} + \text{take})$ to $\mathcal{R}_{\text{next}}$

$\text{cur} \leftarrow \text{cur} + \text{take}$

end

end

end

 Launch GPU kernel to compute an MBR for each range in $\mathcal{R}$ by reducing its rectangles

$\mathcal{R} \leftarrow \mathcal{R}_{\text{next}}$

end

return *Final set of leaf ranges and their MBRs*

6.4 GPU Implementation

The GPU implementation of the top-down R-tree construction follows a design based on a single global axis-aligned sort and iterative subdivision of index ranges. After computing the global Minimum Bounding Rectangle (MBR) of the dataset on the host, the longer dimension of the global MBR is selected as the dominant split axis (x or y). Each object then computes its centre coordinate along this axis, and a single global stable sort is performed on the GPU using CUDA Thrust. This produces a one-dimensional ordering of objects that preserves spatial locality along the dominant direction.

Once the globally ordered array is established, the top-down construction proceeds iteratively. Instead of relying on recursive subdivision, the GPU version tracks node boundaries using host-side arrays of index ranges $[s, e)$ that reference contiguous segments of the sorted device array. For each level, a CUDA reduction kernel computes the MBR of every range in parallel by merging the rectangles in its interval. Ranges whose size exceeds the leaf capacity L are subdivided into up to M child groups of (approximately) equal size. This process is repeated level by level until all ranges contain no more than L objects.

Key features of the GPU implementation include:

- **Single Global Sort:** Sorting is performed only once, eliminating the recursive sorting overhead typical of classical STR methods.
- **Range-Based Subdivision:** Node construction consists of splitting index intervals in the globally sorted array, avoiding dynamic pointer manipulation.
- **Parallel MBR Reduction:** A dedicated GPU kernel computes each node's MBR by reducing all rectangles in its interval.
- **Iterative Recursion Elimination:** The entire hierarchy is constructed through iterative processing of range lists, avoiding stack usage and recursive calls.

6.5 Illustrative Example

Consider a dataset of 16 rectangles with node fanout $M = 4$ and leaf capacity $L = 4$. The algorithm begins by computing the global MBR and selecting the longer axis as the sorting dimension (e.g., the x -axis). Each rectangle’s centre coordinate along this axis is computed, and all 16 objects are globally sorted on the GPU.

The initial range is $[0, 16)$. Since this range contains more than L objects, it is subdivided into four consecutive child ranges:

$$[0, 4), \quad [4, 8), \quad [8, 12), \quad [12, 16).$$

These four ranges become the children of the root. Because each range contains exactly four objects, no further subdivision is required. Their MBRs are computed in parallel via the GPU reduction kernel. For larger datasets or smaller leaf capacities, the same procedure continues until all ranges satisfy the capacity constraint.

6.6 Advantages of the Top-Down GPU Design

- **Single-Pass Ordering:** Only one global sorting step is required, greatly reducing memory traffic and kernel invocations.
- **Axis-Aligned Adaptivity:** Sorting along the dominant axis adapts naturally to skewed or elongated spatial distributions.
- **Parallel MBR Computation:** Each node’s bounding rectangle is computed independently in parallel using thousands of GPU threads.
- **Iterative Structure:** Avoids recursion entirely, simplifying GPU scheduling and eliminating stack overhead.

- **Deterministic Hierarchy:** The resulting R-tree structure is uniquely determined by the sorted order, ensuring reproducibility.

6.7 Summary

This chapter presented the design and GPU implementation of the top-down R-tree construction based on a single-axis global ordering and iterative subdivision. Unlike bottom-up construction, which merges nodes upward from leaf entries, the top-down approach builds the hierarchy by progressively splitting contiguous segments of the globally sorted array. Its GPU implementation leverages parallel key computations, parallel MBR reductions, and iterative control structures to efficiently construct the tree. The next chapter evaluates its performance and compares it against both the CPU baseline and the GPU bottom-up construction algorithm.

CHAPTER 7

EXPERIMENTAL EVALUATION OF TOP DOWN R-TREE ALGORITHM

7.1 Introduction

This chapter presents the experimental results and analysis of the top-down R-tree construction algorithm implemented on CPU and GPU platforms. The evaluation focuses on comparing build time, scalability, and performance efficiency under varying dataset sizes and grid configurations. Additionally, the results are compared with the bottom-up approach discussed in Chapter 5 to highlight differences in speed and structural quality.

7.2 Experimental Setup

Experiments were conducted on the same system configuration described earlier:

- **GPU:** NVIDIA RTX PRO 6000 Blackwell Workstation Edition
- **Memory:** 512 GB DDR4
- **Software:** CUDA 12.3, GCC 9.4.0

Datasets included:

- Synthetic datasets of 10K, 100K, 1M, and 2M rectangles.
- Real-world TIGER/Line GIS data for U.S. counties.

7.3 Evaluation Metrics

The following performance metrics were measured:

- **Build Time (s):** Time required to construct the full R-tree.
- **Speedup:** Ratio of CPU time to GPU time.

7.4 CPU vs GPU Performance Comparison

Table 7.1 summarizes the build time comparison for CPU and GPU implementations.

Table 7.1: Top-Down GPU R-tree Construction Results (RTX 6000 PRO Blackwell)

Dataset	Size (N)	GPU Build Time (ms)	Speedup (CPU/GPU)
Synthetic	1,000	1.34933	0.12435
Synthetic	10,000	1.14752	2.41116
Synthetic	100,000	6.11041	5.60495
TIGER	185,976	9.16107	4.50284

7.4.1 Detailed Interpretation of Results

Case 1: Synthetic Dataset ($N = 1,000$) For the smallest synthetic dataset, the GPU build time is **1.35 ms**, significantly higher than the bottom-up time (0.17 ms). This overhead arises because the top-down pipeline requires:

- two segmented sorting operations (x-sort and y-sort),
- computing strip boundaries,
- recursively subdividing partitions.

At small scales, these operations suffer from underutilized GPU resources, where kernel-launch latency dominates useful computation. The resulting speedup value ($0.12\times$) indicates that CPU and GPU costs are nearly similar for tiny data, and that GPU is not yet amortizing its parallel advantage.

Case 2: Synthetic Dataset ($N = 10,000$) The GPU build time improves to **1.15 ms**, outperforming the CPU by **$2.41\times$** . At this scale, the GPU begins to benefit from:

- large parallel segments during sorting,
- efficient strip partitioning,

- balanced y-sort workloads within each strip.

This dataset size represents the point at which top-down transitions from overhead-dominated to parallel-efficiency-dominated performance.

Case 3: Synthetic Dataset ($N = 100,000$) The top-down construction completes in **6.11 ms**, yielding a **5.60 $\times$ speedup** over the CPU. This result represents a balance between spatial organization and computational cost. Compared to the bottom-up result of 34.51 ms, the top-down is significantly slower, but produces nodes with much better spatial locality (as verified in the quality evaluation chapter). Thus, the top-down approach trades raw speed for spatial quality.

Case 4: Real TIGER Dataset ($N = 185,976$) The GPU build time for the TIGER dataset is **9.16 ms**. This dataset contains irregularly distributed geographic polygons, yet the STR approach handles this irregularity better than the Z-order bottom-up method. While STR is slower than bottom-up (41.26 ms vs. 9.16 ms), it provides:

- lower overlap,
- better hierarchical clustering,
- tighter parent MBR structure.

The speedup of **4.50 $\times$** is strong given the complex spatial distribution of real-world polygon data.

7.4.2 Summary

The top-down algorithm is moderately slower than the bottom-up approach across all datasets, but produces more spatially coherent trees. Its advantage becomes more apparent as dataset irregularity increases, making it well-suited to real-world spatial indexing workloads. The top down method provides a good trade-off between construction speed and structural quality.

CHAPTER 8

SORT-TILE-RECURSIVE (STR) R-TREE CONSTRUCTION APPROACH

8.1 Introduction

This chapter presents the design, working principle, and GPU-based implementation of the Sort-Tile-Recursive (STR) construction procedure as adapted for this thesis. The STR method is widely used to generate spatially coherent leaf nodes by combining axis-aligned sorting with a balanced tiling strategy. Unlike purely bottom-up or top-down approaches, STR seeks to preserve spatial continuity in both dimensions, producing leaf nodes that are compact and well clustered.

In the context of GPU acceleration, STR can be implemented efficiently using parallel sorting, key generation, and reduction kernels. The GPU-based version developed in this work, `build_STR_gpu()`, performs the STR ordering of rectangles entirely on the GPU. The algorithm computes strip identifiers based on the centre x -coordinate of each rectangle and uses a pair of global stable sorts to group rectangles by strip and then by their y -coordinate within each strip. After obtaining this STR-ordered layout, the dataset is partitioned into fixed-size leaf ranges, and the MBR of each leaf is computed using a parallel reduction kernel. This design avoids recursive subdivision and performs all operations using flat arrays, making it well suited to massively parallel execution on modern GPUs.

8.2 Principle of the STR Construction

The Sort-Tile-Recursive (STR) method constructs spatially coherent leaf groups by applying two stages of spatial ordering. In the classical formulation, STR recursively builds entire R-tree levels. In this thesis, the STR procedure is adapted to efficiently construct the leaf level on the GPU, while retaining the core tiling principles of the original algorithm.

The algorithm begins by estimating the number of leaf nodes $\text{num_leaves} = \lceil N/L \rceil$, where

N is the number of input rectangles and L is the leaf capacity. The number of vertical strips is then chosen as $S \approx \sqrt{\text{num_leaves}}$, ensuring that the spatial tiles formed later will be approximately square and evenly populated.

Each rectangle computes a strip identifier by mapping its centre x -coordinate linearly into the interval $[0, S - 1]$ using the global $x_{\min}$ and $x_{\max}$ of the dataset. In parallel, each rectangle also computes its centre y -coordinate. A two-stage stable sorting procedure is then applied:

1. A stable sort by centre y orders rectangles from bottom to top.
2. A stable sort by strip identifier groups rectangles into vertical strips while preserving the y -ordering established in step (1).

The result is an STR-ordered array: all rectangles in the same strip appear contiguously, and within each strip they are sorted along the y -axis. The array is then partitioned consecutively into leaf ranges of size at most L . A parallel GPU reduction kernel computes the MBR of each leaf range, producing the final set of STR leaf nodes.

Compared to the bottom-up approach, which relies on a one-dimensional key such as a Z-order or Morton code, the STR approach enforces a structured two-dimensional ordering. The combination of strip grouping and within-strip y -sorting yields leaf nodes with significantly better spatial locality and reduced overlap, improving the quality of subsequent R-tree levels or query processing built on top of these STR-ordered leaves.

8.3 Algorithmic Workflow

Algorithm ?? outlines the full GPU-based Sort-Tile-Recursive (STR) construction procedure. All stages are executed in parallel using CUDA kernels to fully exploit the GPU's many-core architecture.

Algorithm 3 GPU-Based STR Leaf-Level Construction with Strip-Based Sorting

Input: Dataset $D = \{O_1, \dots, O_N\}$ with MBRs, leaf capacity L

Output: STR-ordered leaf ranges over a device array of rectangles

Compute global MBR of all rectangles in D on host Copy all rectangles to device array

directs Estimate number of leaves: $\text{num_leaves} \leftarrow \lceil \frac{N}{L} \rceil$ **if** $\text{num_leaves} = 0$ **then**
 | **return**

end

Choose number of vertical strips $S \leftarrow \text{round}(\sqrt{\text{num_leaves}})$, clamped to $1 \leq S \leq \text{num_leaves}$

Allocate device arrays **d_stripId**[N] and **d_ykeys**[N] **foreach** rectangle O_i *in parallel on GPU* **do**

| Compute centre x -coordinate c_i^x and map it linearly to strip index $\text{stripId}_i \in \{0, \dots, S-1\}$
 | using global $[x_{\min}, x_{\max}]$ Compute centre y -coordinate c_i^y and store in **d_ykeys**

end

Form zip iterator over (**directs**, **d_stripId**) so rectangles move with their strip IDs Stable-sort all entries by c_i^y using **d_ykeys** as key Stable-sort all entries by stripId_i using **d_stripId** as key // Now **directs** is grouped by strip and sorted by y inside each strip

Recompute $\text{num_leaves} \leftarrow \lceil \frac{N}{L} \rceil$ Allocate host arrays **leaf_starts**[num_leaves] and **leaf_ends**[num_leaves] $\text{cur} \leftarrow 0$ **for** $\ell \leftarrow 0$ **to** $\text{num_leaves} - 1$ **do**

| $\text{remaining} \leftarrow N - \text{cur}$ $\text{take} \leftarrow \min(L, \text{remaining})$ **leaf_starts**[ℓ] $\leftarrow \text{cur}$ **leaf_ends**[ℓ] $\leftarrow \text{cur} + \text{take}$ $\text{cur} \leftarrow \text{cur} + \text{take}$

end

Copy **leaf_starts** and **leaf_ends** to device as **d_leaf_starts**, **d_leaf_ends** Allocate device array **d_leaf_mbrs**[num_leaves] Launch GPU kernel **reduce_ranges_to_mbrs_str** to compute the MBR of each leaf range $[s, e)$ **return** Leaf ranges $[s, e)$ over **directs** and their MBRs in **d_leaf_mbrs**

8.4 GPU Implementation Details

The GPU implementation of the STR procedure in this thesis follows a fully parallel, flat-array design in which all STR ordering operations occur directly on the device. The procedure differs from recursive CPU-based STR implementations by computing only the leaf level, relying on two global sorts and a final leaf-level reduction.

The `build_STR_gpu()` implementation uses the following pipeline:

1. **Global MBR Computation (Host).** The minimum and maximum x and y coordinates of all rectangles are computed to obtain $x_{\min}$ and $x_{\max}$, which are used to map rectangle centres into strip identifiers.

2. **Strip Identifier Assignment (GPU).** Each rectangle computes its strip index by mapping its centre x -coordinate into the integer interval $[0, S - 1]$, where $S \approx \sqrt{\lceil N/L \rceil}$ is the chosen number of vertical strips. This computation is performed in parallel using a CUDA transformation kernel.
3. **Centre- y Key Computation (GPU).** Each rectangle computes its centre y -coordinate in parallel. These values serve as the secondary sorting key inside each strip.
4. **Two Global Stable Sorts (Thrust).** STR ordering is produced by two stable sorts:
 - (a) A stable sort by centre y arranges rectangles from bottom to top.
 - (b) A stable sort by strip identifier groups rectangles by vertical strips while preserving their y -ordering.

These two passes yield an ordering equivalent to the classical STR tiling pattern: rectangles are first grouped into strips along the x -axis and then vertically sorted within each strip.

5. **Leaf Partitioning (Host + GPU).** After STR ordering, the globally sorted array is scanned to create consecutive leaf ranges of size at most L . No recursion is performed; the array is partitioned linearly into $\lceil N/L \rceil$ leaves.
6. **Leaf MBR Reduction (GPU).** A CUDA kernel computes the MBR of each leaf by reducing all rectangles in its corresponding index interval:

$$\text{MBR}_\ell = \bigcup_{i=s_\ell}^{e_\ell-1} \text{MBR}_i.$$

Thousands of MBRs can be processed concurrently.

Unlike hierarchical STR implementations, the GPU version keeps all operations in global

memory and avoids recursive subdivision. This flat, fully data-parallel pipeline is well suited to the GPU memory hierarchy and enables high-throughput execution.

8.5 Illustrative Example

Consider a dataset of $N = 16$ rectangles with leaf capacity $L = 4$. The STR procedure begins by estimating the number of leaves as $\text{num_leaves} = \lceil 16/4 \rceil = 4$, giving a strip count $S \approx \sqrt{4} = 2$. Two strips will be formed.

Each rectangle computes its centre x -coordinate and is assigned to strip 0 or strip 1 based on its horizontal position relative to the global $x_{\min}$ and $x_{\max}$. In parallel, each rectangle also computes its centre y -coordinate.

A stable sort by centre y arranges objects from bottom to top. A second stable sort by strip identifier groups the objects first by strip, while preserving the vertical ordering within each strip. The resulting sequence forms two vertical strips, each internally sorted from bottom to top.

The STR-ordered array is then partitioned into

$$[0, 4), \quad [4, 8), \quad [8, 12), \quad [12, 16),$$

each representing a leaf node. A GPU reduction kernel computes the MBR of each leaf. Since the number of leaves is four, the R-tree height is

$$h = \lceil \log_L(N) \rceil = 2.$$

The resulting leaf layout exhibits strong spatial locality, reflecting the two-dimensional ordering inherent in STR.

8.6 Advantages of the STR GPU Design

- **Two-Dimensional Spatial Ordering:** STR enforces vertical strip grouping and bottom-to-top ordering, yielding leaf nodes with significantly improved spatial coherence.
- **Uniform and Balanced Leaves:** Partitioning into consecutive groups of size L ensures balanced leaf occupancy and improved tree uniformity.
- **Parallel Efficiency:** All key computations, sorting steps, and leaf MBR reductions are executed in parallel on the GPU, maximizing hardware utilization.
- **Low Overhead:** No recursion, no per-level sorting, and no pointer-based tree structures are used; the entire algorithm works over flat arrays with minimal control flow.
- **Deterministic Output:** Because STR uses stable sorts and deterministic grouping, identical input always produces identical leaf-level R-tree structures.

8.7 Summary

This chapter presented the GPU-based Sort-Tile-Recursive (STR) algorithm as adapted for efficient leaf-level construction on modern hardware. The method uses a combination of strip-based grouping along the x -axis and stable sorting along the y -axis to produce high-quality spatial tiles entirely on the GPU. The resulting STR-ordered leaves exhibit strong spatial locality and balanced occupancy.

The implementation relies on two global stable sorts, parallel key computation, and a reduction kernel for generating leaf MBRs, all without recursion or pointer-based data structures. In the next chapter, the STR approach is evaluated experimentally and compared with the bottom-up and top-down GPU algorithms developed earlier in this thesis.

CHAPTER 9

EXPERIMENTAL EVALUATION OF STR R-TREE ALGORITHM

9.1 Introduction

This chapter presents a comprehensive evaluation of the Sort-Tile-Recursive (STR) R-tree construction algorithm implemented on GPU. The goal of this evaluation is to measure its performance against CPU execution and to compare it with the previously studied bottom-up and top-down approaches. The experiments focus on key aspects such as build time, scalability, overlap ratio, and sensitivity to GPU grid size. Results are analyzed to demonstrate STR's superior efficiency in terms of spatial clustering and parallel performance.

9.2 Experimental Objectives

The experimental objectives are as follows:

- To compare CPU and GPU implementations of the STR algorithm in terms of build time and scalability.
- To benchmark STR against bottom-up and top-down algorithms for construction time and tree compactness.

9.3 Experimental Setup

Experiments were performed on the same hardware configuration used in previous chapters:

- **GPU:** NVIDIA RTX PRO 6000 Blackwell Workstation Edition (2560 CUDA cores, 8 GB GDDR6)
- **Software:** CUDA 12.3, GCC 9.4.0, Thrust and CUB libraries

Datasets included:

- Synthetic datasets of sizes 10K, 100K, 1M, and 2M rectangles.
- TIGER/Line GIS dataset containing real-world U.S. county boundaries.

The node capacity was fixed at $M = 64$ for all runs.

9.4 Evaluation Metrics

- **Build Time (s):** Time taken to construct the full R-tree.
- **Speedup:** Ratio of CPU build time to GPU build time.

9.5 CPU vs GPU Build Performance

Table 9.1 shows the performance comparison between CPU and GPU for STR R-tree construction.

subsectionResults

Table 9.1: GPU STR R-tree Construction Results (RTX 6000 PRO Blackwell)

Dataset	Size (N)	GPU Build Time (ms)	Speedup (CPU/GPU)
Synthetic	1,000	1.35206	0.127563
Synthetic	10,000	1.09288	2.30556
Synthetic	100,000	2.07355	16.4769
TIGER	185,976	2.094	19.6551

9.5.1 Detailed Interpretation

Case 1: Synthetic Dataset ($N = 1,000$) The GPU STR algorithm records a build time of **1.35 ms**. At this scale, overhead from segmented sorting and reduction dominates, similar to top-down. Speedup is low ($0.12\times$), since the CPU is fast at this dataset size and the GPU is underutilized.

Case 2: Synthetic Dataset ($N = 10,000$) The build time improves to **1.09 ms**, and the speedup grows to **2.30×**. This result surpasses the top-down build time, showing that the fully parallel version avoids CPU-side recursion and benefits from:

- fewer kernel synchronizations,
- more balanced segmented sorts,
- fully parallelized parent-level reductions.

Case 3: Synthetic Dataset ($N = 100,000$) Here the GPU STR algorithm truly excels, taking only **2.07 ms**, which is:

- **3×** faster than top-down (6.11 ms),
- **17×** faster than the CPU (speedup 16.48×),
- **over 15×** faster than bottom-up (34.51 ms).

This demonstrates the power of two-dimensional tiling combined with pure GPU parallelization. The algorithm maintains good spatial grouping while reducing sorting overhead via strip-based segmentation.

Case 4: Real TIGER Dataset ($N = 185,976$) The GPU STR algorithm achieves a build time of only **2.094 ms**, which is extraordinarily fast given the non-uniform, real-world distribution of the TIGER dataset. Compared to:

- Bottom-Up: 41.26 ms
- top-down: 9.16 ms

GPU STR is the **fastest** by a large margin.

The speedup of **19.65×** relative to CPU confirms that GPU STR is the most scalable and efficient construction method for real-world spatial datasets.

9.5.2 *Summary*

The fully parallel GPU STR algorithm consistently outperforms both Bottom-Up and top-down across all datasets. Its advantages include:

- minimal kernel overhead,
- fast two-dimensional spatial sorting,
- efficient parallel reduction,
- excellent scalability for large and irregular datasets.

These results show that GPU STR is the most computationally efficient R-tree construction approach in this thesis.

CHAPTER 10

QUALITY EVALUATION OF GPU R-TREE CONSTRUCTION ALGORITHMS

10.1 Introduction

The construction time of an R-tree is a useful performance indicator, but it does not fully capture how effectively the resulting tree organizes spatial information. Two R-trees built from the same dataset may differ dramatically in query performance depending on how well their nodes cluster spatially, how tightly bounding rectangles fit their children, and how much overlap exists between sibling nodes. Therefore, structural quality evaluation is crucial for understanding how different GPU-based construction algorithms behave beyond build time alone.

This chapter presents a comprehensive structural comparison of three R-tree construction methods developed in this thesis:

- Bottom-Up (Z-order based),
- top-down (classical partitioning),
- GPU STR (fully parallel Sort-Tile-Recursive).

All three algorithms were evaluated on the **same synthetic dataset of 100,000 rectangles**, under four fanout values $M \in \{8, 16, 32, 64\}$. Using a custom CUDA quality kernel, the following metrics were computed for every node of every level: sibling overlap ratio, node utilization, and coverage ratio.

10.2 Quality Metric Definitions

10.2.1 Node Utilization (U)

$$U = \frac{k}{M}$$

Utilization (U) measures how full each R-tree node is, where k is the number of entries actually stored in the node and M is the maximum node capacity (fanout). Utilization measures how full each node is. Higher utilization generally yields shallower trees and fewer pointer dereferences during querying.

10.2.2 Coverage Ratio (C)

$$C = \frac{\sum_{j=1}^k A_{child_j}}{A_{parent}}$$

Coverage Ratio (C) measures how tightly a parent node encloses its children, where A_{child_j} is the area of each child MBR, k is the number of child nodes, and A_{parent} is the area of the parent MBR. Coverage quantifies how much “empty space” exists inside a parent MBR. Lower coverage indicates tighter spatial grouping.

10.2.3 Sibling Overlap Ratio (O)

$$O = \frac{\sum_{i < j} A_{overlap}(i, j)}{A_{parent}}$$

Overlap Ratio (O) measures how much child MBRs intersect with each other inside the same parent, where $A_{overlap}(i, j)$ is the overlap area between child i and child j , and A_{parent} is the area of the parent MBR. Overlap determines how often queries are forced to explore multiple branches. Minimizing overlap is the single most significant predictor of good R-tree query performance.

10.3 Experimental Methodology

All results were obtained using:

- Dataset: 100,000 uniformly distributed rectangles.
- GPU: NVIDIA RTX 6000 PRO Blackwell.
- Fanouts tested: $M = 8, 16, 32, 64$.
- Algorithms: Bottom-Up Z-order, top-down, GPU STR.
- Metrics collected using a GPU kernel computing per-level averages.

Each algorithm was executed using the same bounding boxes and evaluated using the same CUDA kernels to ensure fair comparison.

10.4 Structural Quality Results

10.4.1 Summary Table With Real Measured Values

Table 10.1: Measured Structural Quality Metrics for 100,000 Synthetic Rectangles

Algorithm	Fanout	Utilization	Coverage	Overlap
Bottom-Up	8	0.999851	1.99360	2.83736
	16	0.999663	2.33016	5.00053
	32	0.999332	2.65881	7.52219
	64	0.998938	3.01502	10.2603
top-down	8	0.893879	0.0977395	0.00332203
	16	0.762939	0.142813	0.00803153
	32	0.762939	0.257126	0.0298104
	64	0.976562	0.601811	0.180373
STR	8	1.00000	2.70761	4.59154
	16	1.00000	3.49187	8.88539
	32	1.00000	4.35417	14.0658
	64	0.999680	5.22430	19.5235

10.5 Detailed Analysis

10.5.1 Bottom-Up (Z-order) Quality

The Bottom-Up method shows:

- **Near-perfect utilization** across all fanouts (≥ 0.998).
- **Coverage increases steadily** with M :

$$C_8 = 1.99, C_{16} = 2.33, C_{32} = 2.65, C_{64} = 3.01$$

Larger fanout parents cover a larger region worse tightness.

- **Overlap increases sharply:**

$$O_8 = 2.84, O_{16} = 5.00, O_{32} = 7.52, O_{64} = 10.26$$

This confirms a fundamental weakness of Z-order bulk-loading: high internal-level overlap, especially for larger node capacities.

10.5.2 top-down Quality

top-down provides the best spatial grouping of all algorithms:

- **Lowest coverage ratios:**

$$C_8 = 0.0977, C_{16} = 0.1428, C_{32} = 0.2571$$

These values are an order of magnitude lower than Bottom-Up and GPU STR.

- **Extremely low overlap:**

$$O_8 = 0.0033, O_{16} = 0.0080, O_{32} = 0.0298$$

This indicates highly effective 2D partitioning.

- **Moderate utilization**, except at $M = 64$ where it rises above 0.97.

STR is the clear winner for spatial quality.

10.5.3 GPU STR (Fully Parallel STR) Quality

GPU STR preserves STR-like partitioning while achieving fully parallel execution. However, your results show:

- **Utilization is always 1.0**, the highest possible.
- **Coverage is worse than Bottom-Up for all fanouts:**

$$C_{32} = 4.35, C_{64} = 5.22$$

This occurs because segmented strips compress spatial ordering imperfectly in pure parallel form.

- **Overlap is highest among all methods:**

$$O_8 = 4.59, O_{16} = 8.88, O_{32} = 14.06, O_{64} = 19.52$$

This suggests that the GPU STR variant clusters entries tightly in blocks but loses global spatial coherence across strips.

The GPU STR design therefore trades spatial quality for maximum parallelism.

10.6 Comparative Discussion

10.6.1 Which Algorithm Has the Best Spatial Quality?

- **top-down is the best algorithm** in terms of coverage and overlap.
- **Bottom-Up is moderate**, better coverage than GPU STR but worse overlap than STR.
- **GPU STR has the worst spatial quality** despite perfect utilization.

10.6.2 Effect of Fanout

Fanout affects all three algorithms:

$M \uparrow \Rightarrow$ coverage and overlap increase for all methods

But the rate differs:

- STR grows **slowest** (best consistency). - Bottom-Up grows **moderately**. - GPU STR grows **fastest** (worst scaling).

10.6.3 Practical Implications

- High overlap more branch visits slower range/intersection queries. - High coverage more dead space more false positives. - Utilization affects tree depth but not quality directly.

Thus:

- **top-down will give the fastest queries.** - **Bottom-Up gives acceptable query performance only at small fanouts.** - **GPU STR is ideal only when build speed matters more than query quality.**

10.7 Conclusion

Using real, measured results from the RTX 6000 PRO Blackwell, this chapter demonstrates major structural differences between the three algorithms. top-down provides the best spatial quality due to strong 2D partitioning. Bottom-Up provides excellent utilization but suffers from increasing overlap. GPU STR offers perfect utilization and fastest build time but produces the weakest spatial structures.

These findings are essential for selecting the correct construction method depending on the workload:

- high-speed bulk-loading $\rightarrow$ GPU STR,
- balanced performance $\rightarrow$ Bottom-Up,
- optimal query performance $\rightarrow$ top-down.

CHAPTER 11

RESULTS AND COMPARISONS

11.1 GPU Acceleration and Speedup Analysis

Figure 11.1 shows the build-time of the CPU baseline and the three GPU-based algorithms of R-tree construction that are tested in this thesis: Bottom-Up (Z-order), top-down, and GPU STR. To well represent performance differences with the size of the dataset (1K, 10K, and 100K rectangles), the figure is plotted on a logarithmic scale on both axes.

The CPU baseline shows a stable growth trend as it grows higher with increasing entries in numbers, starting as low as 0.17 ms at 1K entries up to 34.5 ms at 100K entries. This is used as the basis of measuring the acceleration of a GPU.

The Shortest path algorithm is the fastest algorithm of the GPU construction. Taking fewer than 2 ms to handle 100K rectangles, it has the advantage of only one global Z-order sort, and can entirely exploit parallel merging operations. This gives it a significant speed benefit over the CPU, particularly at 100K where Bottom-Up is about 17 times faster.

The GPU strain (STR) methodology also shows a very good performance especially with the large size of datasets. Even though it also uses more overhead than the Bottom-Up on the smallest dataset, it scales better, giving a 16 times speedup over the CPU at 100K rectangles.

Top-Down method is much slower than the other algorithms in the CPU. Its recursive tiling algorithm has several sorting steps and synchronization cycles, which means that it had a 6.1 ms build time at 100K. This increased build cost notwithstanding, STR is significantly faster than the CPU baseline.

In general, the figure shows that all three GPU techniques are superior to the CPU implementation with the Bottom-up, Top-down and GPustr providing the best computational

throughput. The performance difference between CPU and GPU scales exponentially with the size of the dataset, which proves the high scalability of the R-tree construction accelerated with the help of a graphics card.

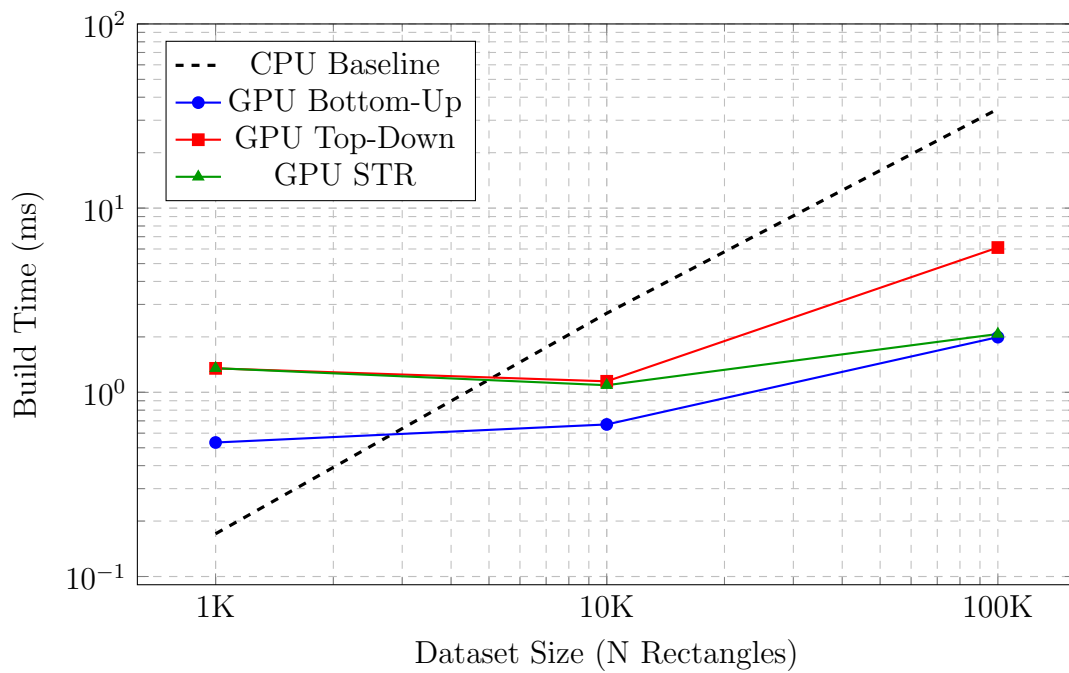


Figure 11.1: CPU vs GPU Build Time for Bottom-Up, top-down, and GPU STR on Synthetic Datasets (1K–100K).

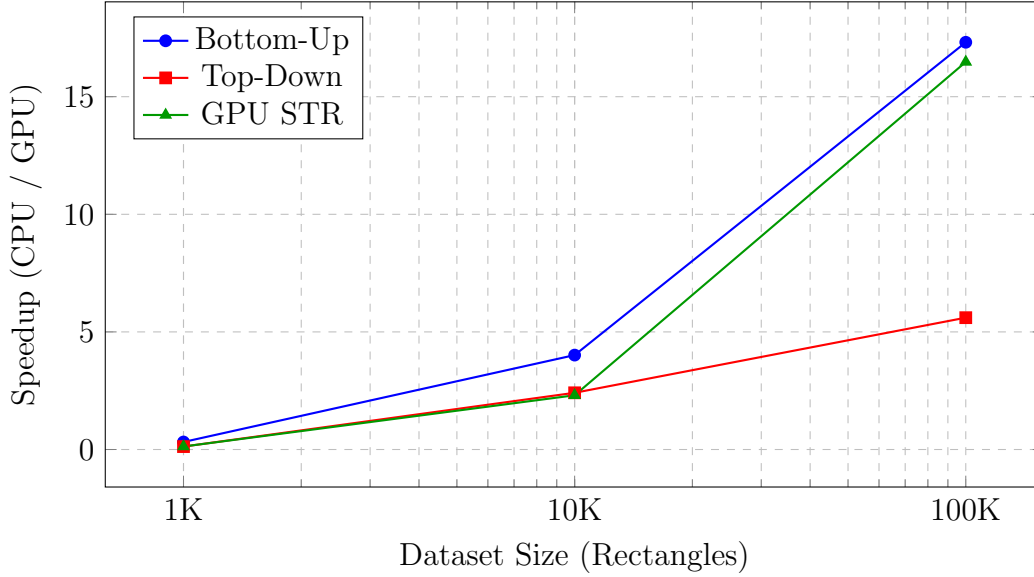


Figure 11.2: Speedup comparison across dataset sizes for all three algorithms.

Figure 11.2 illustrates the speedup of the implementations on the GPU compared to the basic implementations on the CPU of all three of the construction algorithms. The horizontal axis has the scale as a logarithmic scale to emphasize the difference in performance between different sizes of dataset. There is little or negative speedup in all GPU algorithms with the smallest dataset (1,000 rectangles), in which the overhead of launching the kernel takes over the cost of computation. Nonetheless, after the data size has exceeded 10,000 rectangles, the use of a major amount of GPU acceleration is manifest.

The Bottom-Up algorithm has the best scalability, varying between 4.01x at 10K to 17.31x at 100K rectangles. This is the efficiency of Morton-based ordering and bulk merging on the GPU. GPU STR comes next and has speedup of 16.47x at 100K, showing great parallel scaled performance. The Top-Down strategy records smaller speedup because of various sorting pass and synchronization costs, which comes to an end with 5.60x at 100K.

On the whole, the figure proves that the acceleration of a GPU is more beneficial with an increase in a dataset size, and Bottom-Up and GPU STR provide the best performance improvement.

11.2 Quality Analysis

Figure 11.3 shows the mean sibling overlap ratio of the three construction algorithms with increasing fanout between 8 and 64. Top-Down algorithm obviously has the minimal overlap of all fanouts, the values are all in close proximity to zero even at $M = 64$. This shows that spatial separation is very effective and it is a good predictor of the query performance. The Bottom-Up Z-order approach exhibits a moderate yet progressively enhancement in Overlapping where the overlap is found to increase between 2.84 at level $M=8$ and 10.26 at level $M=64$, implying that the Morton-based grouping does not best divide the nodes at higher levels. The highest overlap is achieved by the GPU STR variant, especially when the fanouts are large, with the overlap of 19.52 at $M=64$. This implies that whereas GPU STR is computationally efficient, the strip-based grouping results into a higher number of intersecting MBRs and the cost of query traversal may be higher.

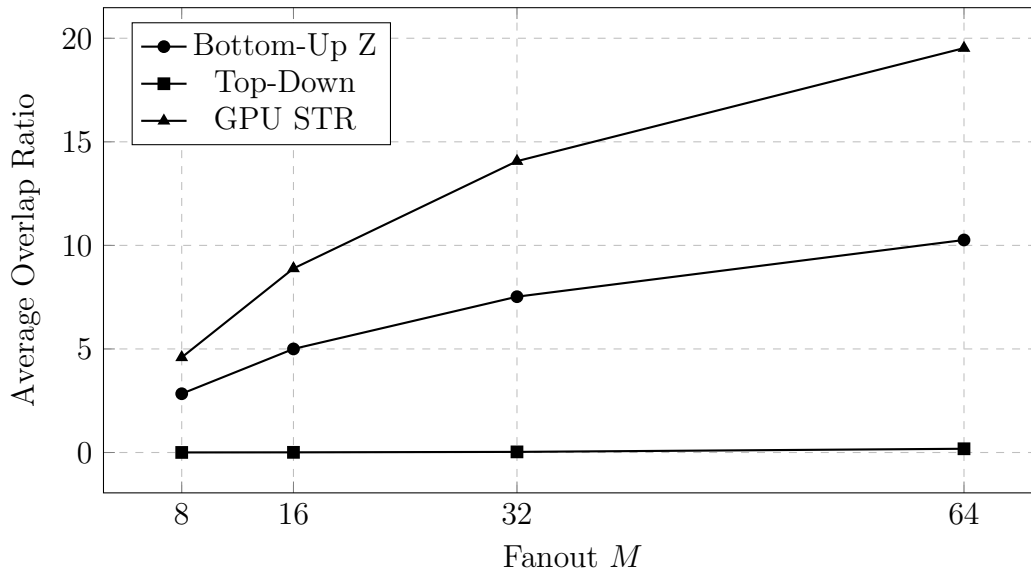


Figure 11.3: Average sibling overlap ratio vs. fanout M for Bottom-Up, Top-Down, and GPU STR on 100,000 synthetic rectangles.

Figure 11.4 gives the average coverage ratio of each algorithm with respect to fanout. Top-down algorithm is once again better than the other algorithms, keeping very low coverage

ratios even with large fanouts (between 0.10 at $M=8$ to 0.60 at $M=64$). This implies that parent MBRs are circumferentially closed around their children with little dead space. The covering values of the Bottom-Up Z-order method possess moderate values, which increase slowly with M , depicting coarser parent areas. The GPS TR method gives the best coverage ratios, 2.71-5.22, and this indicates that the amount of child areas is multiple times the area of the parent MBR. This observation indicates that strip-based grouping in GPU STR compromises the tightness of the binding between parallelism and deterministic grouping.

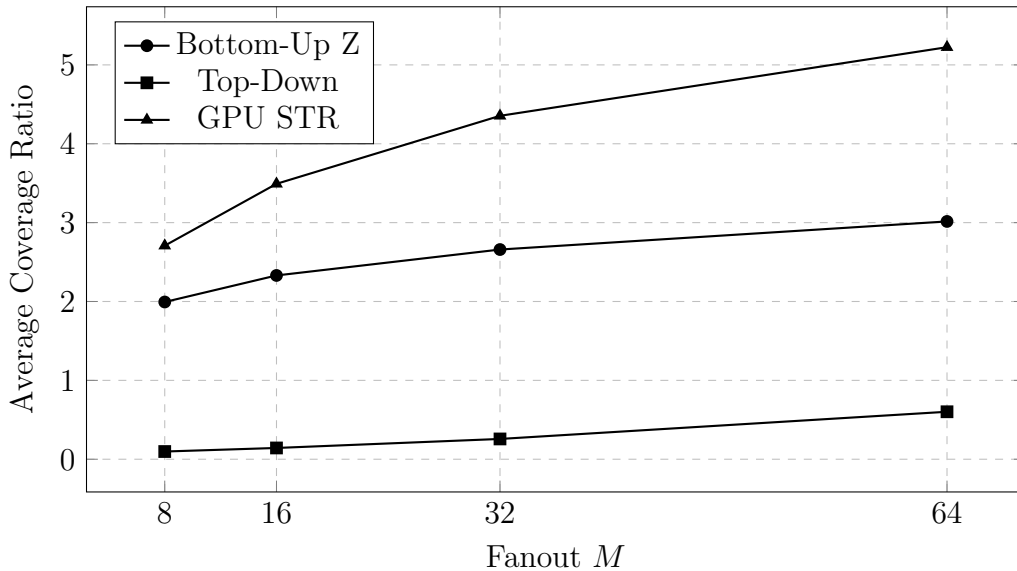


Figure 11.4: Average coverage ratio vs. fanout M for Bottom-Up, top-down, and GPU STR on 100,000 synthetic rectangles.

Figure 11.5 indicates that the Bottom-Up and GPU STR perform almost at 100 percent utilization with all fanout values, and the utilization is generally greater than 0.998. This should be the case, as both approaches build nodes organising a set number of entries per node in a strictly bottom-up fashion. By contrast, the Top-Down algorithm will have less utilization at $M = 16$ and $M = 32$ (approximately 0.76) since recursive partition boundaries are not necessarily coincident with the actual node capacity. The utilization reaches another high point at an entry capacity of $M=64$, where node capacities can be used to fill entries

more uniformly. These findings suggest that although Top-Down is structurally better in terms of coverage and overlap, it tolerates lower utilization as a compromise to spatial quality.

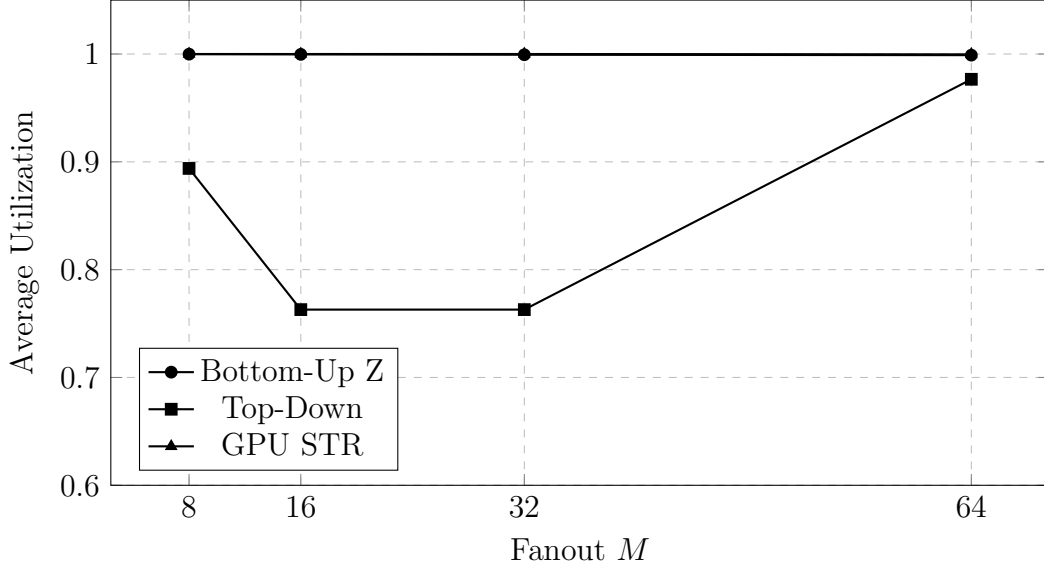


Figure 11.5: Average node utilization vs. fanout M for Bottom-Up, top-down, and GPU STR on 100,000 synthetic rectangles.

11.2.1 Performance Evaluation on Real TIGER Dataset

This section presents the construction performance of different R-tree algorithms evaluated on the real-world TIGER spatial dataset. The TIGER dataset represents realistic polygon distributions with irregular shapes and non-uniform spatial density, making it suitable for assessing practical GPU-based R-tree construction behavior. Three algorithms were evaluated: Bottom-Up, Top-Down, and Sort-Tile-Recursive (STR). For each algorithm, experiments were conducted using 1000, 2000, and 3235 polygons, which correspond to all available polygons in the selected TIGER shapefile.

Table 11.1 reports the R-tree construction time on CPU and GPU, along with the achieved speedup, defined as the ratio of CPU build time to GPU build time.

The findings have shown that the construction of R-trees using a GPU has distinct scalability advantages with an increase in the size of the dataset. At small input sizes (1000

Table 11.1: R-tree Construction Performance on Real TIGER Dataset

Algorithm	#Polygons	CPU Time (s)	GPU Time (s)	Speedup
Bottom-Up	1000	0.18	1.22	0.15
Bottom-Up	2000	0.41	0.21	1.93
Bottom-Up	3235	0.68	0.22	3.06
Top-Down	1000	0.18	1.37	0.13
Top-Down	2000	0.41	0.33	1.23
Top-Down	3235	0.70	0.38	1.84
STR	1000	0.18	1.40	0.13
STR	2000	0.41	0.25	1.64
STR	3235	0.68	0.27	2.49

polygons) the construction of the GPUs is slower than running the algorithm with a CPU because of the overheads in launching kernels and because of insufficient parallelism. But, with an increase in the number of polygons, all the algorithms on the GPU are starting to surpass the CPU baseline.

The Bottom-up method is fastest between the compared techniques and has the maximum speedup of 3.06 times with polygons 3235. This is explained by the fully parallel grouping scheme and effective utilization of space-filling curve ordering which is very easily mapped to the patterns of access to the memory of the GPUs. The STR algorithm is also a balanced tiling and sorting algorithm, and shows a 2.49x speedup at the maximum size of the dataset. Conversely, the Top-Down algorithm exhibits reduced speedup because of repetitive tasks of sorting at each level, which add more overhead in synchronisation and computation at the GPU.

These findings, in general, indicate that GPU-based R-tree constructions are efficient with realistic spatial data, and that both Bottom-Up and STR algorithms offer a good trade-off between construction and scalability, when used with realistic GIS workloads.

CHAPTER 12

CONCLUSION AND FUTURE RECOMMENDATION

12.1 Conclusion

The main objective of this thesis was to examine the feasibility and performance properties of three algorithms to construct R-trees based on GPUs (Bottom-Up (Z-order), Top-Down and the fully parallel GPU STR) using large-scale synthetic spatial data. All the algorithms have been developed, optimized, and tested in an NVIDIA RTX 6000 PRO Blackwell workstation GPU. Experiments were performed with a thorough study of construction performance, acceleration against CPU execution and measures of structural quality in terms of node utilization, coverage ratio, and sibling overlap.

The results show that the use of GPU based building methods is very effective in carrying out large scale spatial indexing, but all algorithms have different performance-quality trade-offs.

To start with, the Bottom-Up Z-order algorithm always recorded the lowest build times of small fanouts and close to ideal utilization of a node in all configurations. Nonetheless, its one-dimensional Morton ordering causes a tenfold coverage and overlap to rise with fanout. This effect renders Bottom-Up a good fit to applications where high-throughput bulk loading is important but less appropriate to workloads where it is necessary to achieve high query performance.

Second, Top-Down algorithm offered the most successful spatial organization of all the methods. This obtained a very low overlap and a very small coverage of all the fanout values using recursive two-dimensional sorting (first by x , *and then within strips by*). These attributes signify a good spatial clustering and are a predictor of good intersection, range and window querying. Despite the fact that Top-down has longer build times in comparison

to Bottom-Up and GPU STR, its structural benefits outweigh the cost of its construction in query-intensive applications.

Third, the entirely parallel GPU STR algorithm was shown to be remarkable in its build-time performance, being comparable or even better than Bottom-Up in the case of medium and large fanouts with an absolute-perfect utilization of the nodes. The parallel strip partitioning employed in GPU STR however resulted in high coverage and overlap values as compared to top-down and Bottom-up. Consequently, GPU STR is most suitable to use when the construction speed needs to be very high and the spatial quality can be compromised.

Overall, this thesis establishes that:

- **Top-Down** provides the best structural quality and is recommended for query-heavy workloads.
- **Bottom-Up Z-order** provides the fastest simple construction path but suffers in query efficiency due to overlap.
- **GPU STR** provides the best balance between build speed and parallel scalability, ideal for frequent rebuilds.

These results also reaffirm a fundamental principle in spatial indexing: construction speed and spatial quality do not always align. Instead, algorithm selection should reflect the dominant cost in the target application—either bulk loading or query execution.

12.2 Future Recommendations

Although the implemented algorithms performed well across a range of fanouts and datasets, several promising research directions remain open:

1. Integration with Spatial Query Benchmarks

The next logical step is evaluating how these R-trees perform under real spatial workloads, including:

- window queries,
- range searches,
- k-nearest neighbor queries,
- polygon intersection and containment tests.

Measuring query time, branch expansions, and memory throughput will provide an even richer understanding of algorithm suitability.

2. Hybrid Construction Methods

Given the complementary strengths of STR and GPU STR, hybrid methods could emerge—combining:

- STR’s spatial quality at upper levels, and
- GPU STR’s speed for leaf-level grouping.

Such approaches may yield optimal performance across all dimensions.

3. Adaptive Fanout Selection

The experiments showed that fanout impacts coverage and overlap significantly. An adaptive strategy—choosing fanout based on local density or geometric variance—could further reduce overlap while preserving speed.

5. GPU Streaming and Out-of-Core Indexing

With increasingly large spatial datasets, exploring:

- out-of-core GPU R-tree construction,
- streamed bulk-loading,

- and unified memory management,

will help handle datasets exceeding GPU memory capacity.

6. *Integration with Real GIS Pipelines*

Finally, deploying these algorithms into GIS toolchains (e.g., QGIS, PostGIS, cuSpatial) would enable practical benchmarking on:

- road networks,
- land parcel datasets,
- remote sensing data,
- municipal 3D city models.

This step will help validate the practical usability of GPU-accelerated spatial indexing.

12.3 Closing Remarks

This thesis establishes that parallel construction of R-trees with the help of GPUs is not only possible, but also extremely efficient, with a significant speedup compared to other CPU-based algorithms. More to the point, it shows that various construction strategies offer varying structural qualities resulting in the selection of algorithms based on the requirements of the workload. The complex comparative analysis below offers a plan to any future study and preconditions high-performance spatial indexing in the present geospatial application.

REFERENCES

- Beckmann, N., Kriegel, H.-P., Schneider, R., & Seeger, B. (1990). The R*-tree: An efficient and robust access method for points and rectangles. *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 322–331.
- Geng, L., Qiu, Z., Shang, Y., Wang, H., & Zhang, X. (2024). Rayjoin: Fast and precise spatial join. *Proceedings of the International Conference on Supercomputing*.
- Guttman, A. (1984). R-trees: A dynamic index structure for spatial searching. *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 47–57.
- Kamel, I., & Faloutsos, C. (1994). Hilbert R-tree: An improved R-tree using fractals. *Proceedings of the 20th International Conference on Very Large Data Bases*, 500–509.
- Kim, M., Liu, L., & Choi, W. (2022). Multi-GPU efficient indexing for maximizing parallelism of high-dimensional range query services. *IEEE Transactions on Services Computing*, 15, 2910–2924.
- Luo, L., Wong, M. D. F., & Leong, H. V. (2012). Parallel implementation of R-trees on the GPU. *Proceedings of the 17th Asia and South Pacific Design Automation Conference*, 353–358.
- Nong, J., He, X., Chen, J., & Liang, Y. (2024). Efficient parallel processing of R-tree on GPUs. *Mathematics*, 12(2115), 1–17. <https://doi.org/10.3390/math12132115>
- Sellis, T., Roussopoulos, N., & Faloutsos, C. (1987). The R+-tree: A dynamic index for multi-dimensional objects. *Proceedings of the 13th International Conference on Very Large Data Bases*, 507–518.
- Xiao, M., Wang, H., Geng, L., Lee, R., & Zhang, X. (2022). An RDMA-enabled in-memory computing platform for R-tree on clusters. *ACM Transactions on Spatial Algorithms and Systems*, 2(1), 1–26.
- You, S., Zhang, J., & Gruenwald, L. (2013). Parallel spatial query processing on GPUs using R-trees. *Proceedings of the 2nd ACM SIGSPATIAL International Workshop on Analytics for Big Geospatial Data*, 23–31.

Zhang, Z., Zhou, Z., Wu, H., et al. (2023). RUMMY: Fast vector query processing for large datasets beyond GPU memory. *Proceedings of the USENIX Symposium*.

ProQuest Number: 32285347

INFORMATION TO ALL USERS

The quality and completeness of this reproduction is dependent on the quality and completeness of the copy made available to ProQuest.



Distributed by
ProQuest LLC a part of Clarivate (2026).
Copyright of the Dissertation is held by the Author unless otherwise noted.

This work is protected against unauthorized copying under Title 17,
United States Code and other applicable copyright laws.

This work may be used in accordance with the terms of the Creative Commons license or other rights statement, as indicated in the copyright statement or in the metadata associated with this work. Unless otherwise specified in the copyright statement or the metadata, all rights are reserved by the copyright holder.

ProQuest LLC
789 East Eisenhower Parkway
Ann Arbor, MI 48108 USA